

Dict data type

A collection of key-value pairs enclosed within curly brackets {} separated by commas is called a Dictionary.

So we can say dictionary is a collection of key-value pairs.

1. Collection of key-value pairs enclosed within {} is called a dictionary.
2. Duplicate keys are NOT allowed but values can be duplicated.
3. Unordered collection (accessed using keys).
4. Indexing not supported (keys are used instead).
5. Slicing not supported.
6. Different data types allowed.
7. Mutable in nature (values can be modified).

Examples:

```
d1 = {"name": "Rahul", "age": 25, "course": "Python"}
d2 = {1: "Java", 2: "Python", 3: "C++"}
d3 = {"a": 10, "b": 20, "c": 30}
```

```
print(d1)
print(d2)
print(d3)
```

Output

```
{'name': 'Rahul', 'age': 25, 'course': 'Python'}
{1: 'Java', 2: 'Python', 3: 'C++'}
{'a': 10, 'b': 20, 'c': 30}
```

Accessing Dictionary Elements:-

```
d = {"name": "Rahul", "age": 25}
```

```
print(d["name"])
print(d["age"])
```

Output

```
Rahul
25
```

Modifying Dictionary Values:-

```
d = {"name": "Rahul", "age": 25}
```

```
d["age"] = 26
```

```
print(d)
```

Output

```
{'name': 'Rahul', 'age': 26}
```

Adding New Elements:-

```
d = {"name": "Rahul", "age": 25}
```

```
d["course"] = "Python"
```

```
print(d)
```

Output

```
{'name': 'Rahul', 'age': 25, 'course': 'Python'}
```

Python In-built functions for Dictionary:-

Step	Syntax	Explanation
1	len()	To count number of key-value pairs
2	type()	To check datatype of object
3	id()	To check memory address
4	dict()	To create empty dictionary or type casting
5	print()	To print output

Examples:-

```
d = {"a": 10, "b": 20, "c": 30}
```

```
print(len(d))
```

```
print(type(d))
```

```
print(id(d))
```

Output

```
3
```

```
<class 'dict'>
```

```
135446907837040
```

Dictionary Methods

1. keys()

2. values()

3. items()
4. get()
5. update()
6. pop()
7. popitem()
8. clear()
9. copy()
10. setdefault()
11. fromkeys()

1. keys():-- Returns all keys:-

```
d = {"a": 10, "b": 20, "c": 30}

print(d.keys())

# O/P: dict_keys(['a', 'b', 'c'])
```

2. values():-- Returns all values:-

```
d = {"a": 10, "b": 20, "c": 30}

print(d.values())

# O/P: dict_values([10, 20, 30])
```

3. items():-- Returns key-value pairs:-

```
d = {"a": 10, "b": 20}

print(d.items())

# O/P: dict_items([('a', 10), ('b', 20)])
```

4. get():-- Access value using key:-

```
d = {"name": "Rahul", "age": 25}

print(d.get("name"))

# O/P: Rahul
```

5. update():-- Update dictionary:-

```
d = {"a": 10, "b": 20}

d.update({"b": 50, "c": 30})

print(d)
```

```
# O/P: {'a': 10, 'b': 50, 'c': 30}
```

6. pop():-- Remove element using key:-

```
d = {"a": 10, "b": 20}
d.pop("a")
print(d)
```

```
# O/P: {'b': 20}
```

7. popitem():-- Remove last inserted item:-

```
d = {"a": 10, "b": 20}
d.popitem()
print(d)
```

```
# O/P: {'a': 10}
```

8. clear():-- Remove all elements:-

```
d = {"a": 10, "b": 20}
d.clear()
print(d)
```

```
# O/P: {}
```

9. copy():-- Copy dictionary:-

```
d1 = {"a": 10, "b": 20}
d2 = d1.copy()
print(d2)
```

```
# O/P: {'a': 10, 'b': 20}
```

10. setdefault():-- Returns value of key, if not present creates it:-

```
d = {"a": 10, "b": 20}
d.setdefault("c", 30)
print(d)
```

```
# O/P: {'a': 10, 'b': 20, 'c': 30}
```

11. fromkeys():-- Creates a new dictionary with given keys and same value:-

```
keys = ("a", "b", "c")
d = dict.fromkeys(keys, 10)
print(d)
```

O/P:

```
{'a': 10, 'b': 10, 'c': 10}
```

```
keys = ["name", "age", "course"]
```

```
d = dict.fromkeys(keys)
```

```
print(d)
```

O/P:

```
{'name': None, 'age': None, 'course': None}
```